LSI S5 — Cloud Computing

Rapport de TP — Assignment 3

Simulation d'infrastructures cloud avec
CloudSim 7.0.1 et CloudSim Plus 8.5.7

Réalisé par : **Khazri Salah Eddine**

Encadré par : **Prof. C. El Amrani**

Octobre 2026

Sommaire

Introduction et environnement de travail 3
Environnement 3
Organisation du code 3
Partie 1 — CloudSim 7.0.1 4
Exercice 1 : coût des cloudlets 4
Exercice 2 : hôtes hétérogènes et temps d'exécution 5
Exercice 3 : plusieurs datacenters et plusieurs utilisateurs 6
Exercice 4 : ordonnancement time-shared vs space-shared 8
Partie 2 — CloudSim Plus 8.5.7 10
Exercice 5 : de CloudSim 7 à CloudSim Plus 10
Exercice 6 : politiques d'allocation de VMs 12
Exercice 7 : brokers et répartition de charge 14
Conclusion 17

Introduction et environnement de travail

Ce rapport présente la réalisation des sept exercices de l'Assignment 3. La première partie utilise **CloudSim 7.0.1** (le dépôt fourni, compilé localement) et la seconde **CloudSim Plus 8.5.7** (téléchargé depuis Maven Central). Pour chaque exercice, je donne la configuration simulée, les résultats obtenus (copiés des sorties du programme) et l'analyse demandée.

Environnement

Élément	Version / détail
Système	Windows 11 Pro
JDK	Java 21 (javac / java 21)
Build	Apache Maven 3.9.12
CloudSim	7.0.1 — modules `cloudsim` et `cloudsim-examples` du dépôt, réinstallés avec `mvn install`
CloudSim Plus	8.5.7 (`org.cloudsimplus:cloudsimplus:8.5.7`) ; exemples tirés du dépôt `cloudsimplus-examples`

Organisation du code

Tout le code du TP se trouve dans le dossier `TP3/` à la racine du dépôt :

Fichier	Rôle
`TP3/cloudsim7/…/tp3/Cs7Util.java`	Fonctions communes CloudSim 7 (création d'hôtes, datacenters, VMs, cloudlets)
`RecordingBroker.java`	DatacenterBroker qui mémorise le datacenter et l'hôte de chaque VM
`VmAllocationPolicyFixed.java`	Politique plaçant la VM i sur l'hôte i (Ex. 2)
`CostAwareBroker.java`	Broker qui choisit d'abord le datacenter le moins cher (Ex. 3)
`Ex1Costs`, `Ex2Heterogeneous`, `Ex3MultiDatacenters`, `Ex4Schedulers`	Exercices 1 à 4 (CloudSim 7)
`TP3/cloudsimplus/…/tp3plus/BasicFirstExample.java`	Copie de l'exemple officiel (Ex. 5 Q1)
`Ex5Heterogeneous`, `Ex5CostsEx1`	Exercice 5 (réécriture de l'Ex. 2 et coûts)
`Ex6AllocationPolicies`, `Ex7Brokers`	Exercices 6 et 7 (CloudSim Plus)
`TP3/outputs/*.txt`	Sorties complètes de toutes les exécutions

Commandes d'exécution (depuis `TP3/cloudsim7` ou `TP3/cloudsimplus`) :

```
mvn -q compile dependency:build-classpath -Dmdep.outputFile=cp.txt
java -cp "target/classes;$(cat cp.txt)" tp3.Ex1Costs            # CloudSim 7
java -cp "target/classes;$(cat cp.txt)" tp3plus.Ex6AllocationPolicies   # CloudSim Plus
```

> **Remarque préalable :** le jar CloudSim installé dans le dépôt Maven local (`~/.m2`) était plus ancien que les sources du dépôt (le dernier commit ajoute des constructeurs sans identifiant). L'exemple 2 original plantait avec `NoSuchMethodError: Pe.<init>(PeProvisioner)`. J'ai donc relancé `mvn install -DskipTests` à la racine avant de commencer.

Partie 1 — CloudSim 7.0.1

Exercice 1 : coût des cloudlets

Base : `CloudSimExample2` — un datacenter, un hôte de 1 PE à 1000 MIPS (2048 Mo de RAM, ordonnanceur `VmSchedulerTimeShared`), des VMs de 250 MIPS / 512 Mo, et un cloudlet lié à chaque VM. Coûts du datacenter : **3,0 $/s** de CPU, 0,05 $/Mo de mémoire, 0,001 $/Mo de stockage, **0,0 $/Mo** de bande passante. Le code se trouve dans `Ex1Costs.java`.

Q1. Exécution de l'exemple inchangé

Cloudlet	Statut	DC	VM	Longueur (MI)	Temps CPU (s)	Début (s)	Fin (s)
0	SUCCESS	2	0	250 000	1000	0,01	1000,01
1	SUCCESS	2	1	250 000	1000	0,01	1000,01

Tableau 1 — Sortie de CloudSimExample2 (outputs/Ex1_CloudSimExample2_original.txt)

Les deux cloudlets commencent à 0,01 s (délai de création des VMs et de soumission) et se terminent à 1000,01 s : 250 000 MI ÷ 250 MIPS = 1000 s. Les deux VMs (2 × 250 = 500 MIPS) tiennent sur le PE de 1000 MIPS, donc chaque VM reçoit bien ses 250 MIPS et les deux cloudlets avancent en parallèle.

Q2 et Q3. Trois cloudlets de longueurs différentes et colonne « Cost »

J'ai créé 3 VMs de 250 MIPS (3 × 250 = 750 ≤ 1000 MIPS, 3 × 512 = 1536 ≤ 2048 Mo) et 3 cloudlets de 40 000, 80 000 et 120 000 MI, chacun lié à sa VM avec `cloudlet.setGuestId(vmId)`. La colonne **Cost** affiche `cloudlet.getProcessingCost()`, et j'ai ajouté une colonne **CPU cost** calculée par `getActualCPUTime() × getCostPerSec()`.

Cloudlet	VM	Longueur	Temps CPU	Début	Fin	Cost (getProcessingCost)	CPU cost
0	0	40 000	160,00	0,01	160,01	0,00	480,00
1	1	80 000	320,00	0,01	320,01	0,00	960,00
2	2	120 000	480,00	0,01	480,01	0,00	1440,00
Total						**0,00**	**2880,00**

Tableau 2 — 3 cloudlets, coût CPU de 3 $/s, coût de bande passante nul

Q4. Calcul manuel du coût CPU et comparaison

Le coût CPU attendu est : **temps CPU réel × coût par seconde**.

- Cloudlet 0 : 40 000 / 250 = 160 s → 160 × 3,0 = **480 $**
- Cloudlet 1 : 80 000 / 250 = 320 s → 320 × 3,0 = **960 $**
- Cloudlet 2 : 120 000 / 250 = 480 s → 480 × 3,0 = **1440 $**

Le calcul manuel correspond à la colonne « CPU cost », mais **la colonne « Cost » affiche 0,00**. Pour comprendre, j'ai lu le code de `Cloudlet.getProcessingCost()` dans CloudSim 7.0.1 :

```
public double getProcessingCost() {
    // cloudlet cost: execution cost...
    // double cost = getProcessingCost();      <-- partie CPU commentée !
    double cost = 0;
    cost += accumulatedBwCost;                // costPerBw x taille du fichier d'entrée
    cost += costPerBw * getCloudletOutputSize(); // costPerBw x taille du fichier de sortie
    return cost;
}
```

Explication de la différence : dans CloudSim 7, `getProcessingCost()` ne contient **que le coût de transfert des données** : costPerBw × (fileSize + outputSize). La partie CPU est commentée et les coûts mémoire et stockage ne sont jamais facturés au cloudlet. Comme `costPerBw = 0,0` dans l'exemple 2, la valeur affichée est 0. Pour le

vérifier, j'ai relancé avec `costPerBw = 0,1 $/Mo` : chaque cloudlet affiche alors 0,1 × (300 + 300) = **60 $**, soit exactement le transfert, quelle que soit sa longueur (tableau 3).

Cloudlet	Longueur	Temps CPU	Cost (getProcessingCost)	CPU cost (manuel)	Coût total
0	40 000	160	60,00	480,00	540,00
1	80 000	320	60,00	960,00	1020,00
2	120 000	480	60,00	1440,00	1500,00
Total			180,00	2880,00	**3060,00**

Tableau 3 — Même scénario avec costPerBw = 0,1 $/Mo

Le coût réel d'un cloudlet doit donc être reconstitué ainsi : **coût = temps CPU × costPerSec + costPerBw × (fileSize + outputSize)**. À noter aussi : `getActualCPUTime()` vaut `finishTime − startTime`, c'est-à-dire un temps « mur » et non un vrai temps CPU. Si plusieurs cloudlets partagent une VM en time-shared, chacun est facturé pour toute la durée où il est présent, y compris le temps passé à attendre le processeur.

Q5. Coût total, cloudlet le plus cher, doublement du prix

- **Coût total pour l'utilisateur** (CPU, avec costPerBw = 0) : 480 + 960 + 1440 = **2880 $**.
- **Cloudlet le plus cher : le cloudlet 2** (1440 $, soit 50 % du total). Toutes les VMs ont la même puissance (250 MIPS) et le même prix (3 $/s), donc le coût est proportionnel au temps d'exécution, lui-même proportionnel à la longueur (120 000 MI, 3 fois plus que le cloudlet 0).
- **Coût par seconde doublé (6,0 $/s)** : 960 $, 1920 $ et 2880 $, soit un total de **5760 $**. Le coût CPU double exactement : il est linéaire en costPerSec, alors que le temps d'exécution ne change pas. Seule la partie bande passante (si elle n'est pas nulle) resterait identique.

Exercice 2 : hôtes hétérogènes et temps d'exécution

Base : `CloudSimExample3`. Un datacenter avec les 4 hôtes de l'énoncé (`VmSchedulerTimeShared`), une VM par hôte et des VMs en `CloudletSchedulerTimeShared` (code : `Ex2Heterogeneous.java`).

Hôte	PEs	MIPS / PE	RAM (Mo)	VM créée dessus
Host 0	1	1000	2048	VM 0 : 1 PE, 1000 MIPS
Host 1	2	1500	4096	VM 1 : 1 PE, 1500 MIPS
Host 2	4	2000	8192	VM 2 : 2 PEs, 2000 MIPS
Host 3	2	3000	4096	VM 3 : 1 PE, 3000 MIPS

> **Problème rencontré :** avec la politique par défaut `VmAllocationPolicySimple` (sélection « LeastFull » : l'hôte ayant le plus de PEs libres), les VMs ne sont pas placées sur l'hôte prévu. La VM 3 (3000 MIPS par PE) ne pouvait plus être créée, car le seul hôte capable de l'accueillir (Host 3) avait déjà été pris par une autre VM, et les cloudlets qui lui étaient liés n'étaient jamais exécutés. J'ai donc écrit `VmAllocationPolicyFixed`, qui place la VM i sur l'hôte i comme le demande le tableau.

Q1 et Q2. Affectation initiale et tableau des résultats

Les 5 cloudlets (1 PE chacun) sont affectés avec `cloudlet.setGuestId(i mod 4)`, ce qui reproduit le round-robin du broker : C0→VM0, C1→VM1, C2→VM2, C3→VM3, C4→VM0.

Cloudlet	Longueur	VM	MIPS VM	Début	Fin	Temps exéc.	Longueur ÷ MIPS
0	30 000	0	1000	0,01	60,01	60,00	30,00
1	60 000	1	1500	0,01	40,01	40,00	40,00
2	90 000	2	2000	0,01	45,01	45,00	45,00
3	120 000	3	3000	0,01	40,01	40,00	40,00
4	150 000	0	1000	0,01	180,01	**180,00**	150,00

Tableau 4 — Affectation initiale (makespan = 180,01 s)

Q3. Explication et makespan

Pour les cloudlets 1, 2 et 3, on a bien **temps d'exécution = longueur ÷ MIPS**, car chacun est seul sur sa VM. Les cloudlets 0 et 4 font exception (lignes rouges) : ils partagent la VM 0, la plus lente, en mode time-shared.

- De 0 à 60 s, chacun reçoit 1000 / 2 = 500 MIPS. C0 termine à 30 000 / 500 = **60 s** (au lieu de 30 s).
- À t = 60 s, C4 a exécuté 30 000 MI. Il lui reste 120 000 MI à 1000 MIPS, soit 120 s de plus : fin à **180 s** (au lieu de 150 s).

Makespan = 180,01 s. La charge est très mal répartie : la VM la plus lente (1000 MIPS) reçoit 180 000 MI, alors que la plus rapide (3000 MIPS) n'en reçoit que 120 000 et reste inactive à partir de 40 s. De même, la VM 2 a 2 PEs mais un seul cloudlet (1 PE) : son second PE ne sert à rien.

Q4. Optimisation : heuristique LJF

J'ai implémenté LJF (Longest Job First) avec le choix de la ressource qui termine le plus tôt. On trie les cloudlets par longueur décroissante, puis chacun est placé sur le « slot » (PE de VM) qui le terminerait au plus tôt, compte tenu de ce qui y est déjà affecté. La VM 2 offre 2 slots de 2000 MIPS.

Ordre LJF	Cloudlet	Choix	Fin estimée
1	C4 — 150 000 MI	VM 3 (3000 MIPS)	50 s
2	C3 — 120 000 MI	VM 2, PE 1 (2000 MIPS)	60 s (sur VM 3 : 90 s)
3	C2 — 90 000 MI	VM 2, PE 2 (2000 MIPS)	45 s
4	C1 — 60 000 MI	VM 1 (1500 MIPS)	40 s
5	C0 — 30 000 MI	VM 0 (1000 MIPS)	30 s

Cloudlet	Longueur	VM	MIPS VM	Début	Fin	Temps exéc.	Longueur ÷ MIPS
0	30 000	0	1000	0,01	30,01	30,00	30,00
1	60 000	1	1500	0,01	40,01	40,00	40,00
2	90 000	2	2000	0,01	45,01	45,00	45,00
3	120 000	2	2000	0,01	60,01	60,00	60,00
4	150 000	3	3000	0,01	50,01	50,00	50,00

Tableau 5 — Affectation LJF simulée (makespan = 60,01 s)

Comparaison : le makespan passe de **180,01 s à 60,01 s (−66,7 %)**. Désormais, chaque cloudlet vérifie temps = longueur ÷ MIPS, puisqu'aucun ne partage un PE. Ce résultat est même optimal : le cloudlet de 120 000 MI demande au moins 40 s sur la VM 3, mais s'il y est placé, celui de 150 000 MI doit aller sur la VM 2 (75 s). Le meilleur compromis donne 60 s, ce que trouve LJF. La borne inférieure théorique, 450 000 MI ÷ 9500 MIPS = 47,4 s, ne peut pas être atteinte car les cloudlets sont indivisibles.

Exercice 3 : plusieurs datacenters et plusieurs utilisateurs

Base : `CloudSimExample4` (deux datacenters) et `CloudSimExample5` (deux utilisateurs). Code : `Ex3MultiDatacenters.java`.

Q1 et Q2. Configuration

- **3 datacenters** (Datacenter_0, _1, _2), chacun avec **2 hôtes** de 2 PEs à 1000 MIPS (4096 Mo, `VmSchedulerTimeShared`). Coût CPU respectif : **3,0, 2,0 et 1,0 $/s**.
- **3 utilisateurs** (Broker_0, Broker_1, Broker_2), chacun avec **2 VMs** (1 PE, 1000 MIPS, 1024 Mo) et **3 cloudlets** de 20 000, 40 000 et 60 000 MI.

- Initialisation avec le bon nombre d'utilisateurs : `CloudSim.init(3, calendar, false)`.

Q3. Résultats : datacenter et VM de chaque cloudlet

Utilisateur	Cloudlet	VM	Datacenter / Hôte	Longueur	Début	Fin	Coût ($)
Broker_0	0	0	Datacenter_0 / Host 0	20 000	0,01	40,01	120
Broker_0	1	1	Datacenter_0 / Host 1	40 000	0,01	40,01	120
Broker_0	2	0	Datacenter_0 / Host 0	60 000	0,01	80,01	240
Broker_1	3	2	Datacenter_0 / Host 0	20 000	0,01	40,01	120
Broker_1	4	3	Datacenter_0 / Host 1	40 000	0,01	40,01	120
Broker_1	5	2	Datacenter_0 / Host 0	60 000	0,01	80,01	240
Broker_2	6	4	Datacenter_1 / Host 0	20 000	0,02	40,02	80
Broker_2	7	5	Datacenter_1 / Host 1	40 000	0,02	40,02	80
Broker_2	8	4	Datacenter_1 / Host 0	60 000	0,02	80,02	160

Tableau 6 — Brokers par défaut (coût total : 480 + 480 + 320 = 1280 $, makespan 80,02 s)

Quels datacenters reçoivent des VMs ? Seulement Datacenter_0 et Datacenter_1. Datacenter_2, pourtant le moins cher, ne reçoit rien. L'explication vient du code de `DatacenterBroker` :

1. Après avoir reçu les caractéristiques de tous les datacenters, le broker appelle `createVmsInDatacenter(getDatacenterIdsList().getFirst())` : il demande la création de **toutes** ses VMs dans le **premier** datacenter de la liste, c'est-à-dire dans l'ordre d'enregistrement et non selon le prix.
2. Il ne passe au datacenter suivant que pour les VMs dont la création a échoué.
3. Chaque datacenter offre 2 hôtes × 2 PEs = 4 PEs, donc 4 VMs de 1 PE. Les 4 VMs de Broker_0 et Broker_1 remplissent Datacenter_0. Les VMs de Broker_2 y sont refusées et vont dans Datacenter_1. Datacenter_2 n'est jamais sollicité.

Par ailleurs, les cloudlets non liés sont distribués en round-robin sur les VMs créées : avec 3 cloudlets pour 2 VMs, la première VM exécute 2 cloudlets en time-shared (20 000 et 60 000 MI), d'où une fin à 80 s. Enfin, Broker_0 et Broker_1 paient 3 $/s alors qu'un datacenter à 1 $/s reste libre.

Q4. Optimiser le temps, minimiser le coût : deux objectifs compatibles ?

J'ai simulé deux optimisations, séparément puis ensemble :

- **Temps (LJF) :** pour chaque utilisateur, le cloudlet le plus long (60 000 MI) est seul sur la VM 0, et les deux autres (20 000 + 40 000) partagent la VM 1. Chaque VM a alors 60 000 MI à traiter : 60 s, ce qui est la borne optimale (120 000 MI ÷ 2000 MIPS).
- **Coût (`CostAwareBroker`) :** ce broker trie la liste des datacenters par `getCostPerSecond()` croissant avant de créer ses VMs. Il essaie donc Datacenter_2 (1 $/s), puis Datacenter_1 (2 $/s), puis Datacenter_0.

Scénario	DCs utilisés	Makespan	Coût B0	Coût B1	Coût B2	Coût total
A — Défaut	DC0 (B0, B1), DC1 (B2)	80,02 s	480 $	480 $	320 $	1280 $
B — LJF (temps)	DC0, DC1	**60,02 s**	480 $	480 $	320 $	1280 $
C — Moins cher (coût)	DC2 (B0, B1), DC1 (B2)	80,02 s	160 $	160 $	320 $	**640 $**
D — LJF + moins cher	DC2, DC1	**60,02 s**	160 $	160 $	320 $	**640 $**

Tableau 7 — Comparaison des optimisations (outputs/Ex3MultiDatacenters.txt)

Analyse :

- **Optimiser le temps :** répartir les cloudlets selon leur longueur (LJF/MCT) et la puissance des VMs, utiliser plus de VMs ou des VMs plus rapides, et éviter de cumuler plusieurs cloudlets sur une VM pendant qu'une autre est inactive. Le makespan passe de 80 à 60 s (−25 %).

- **Minimiser le coût :** choisir le datacenter au coût par seconde le plus faible et éviter le temps-mur facturé inutilement. Le coût total passe de 1280 à 640 $ (−50 %). Broker_0 et Broker_1 passent de 480 à 160 $ chacun.
- **Compatibilité :** dans ce scénario, les deux objectifs sont compatibles (scénario D), car toutes les VMs ont la même puissance quel que soit le datacenter. Le coût dépend du choix du datacenter, le temps de l'affectation des cloudlets : les deux leviers sont indépendants.
- **En général, ils sont en conflit.** (1) Le datacenter le moins cher a une capacité limitée : Broker_2 arrive trop tard, ne trouve plus de place dans Datacenter_2 et paie 2 $/s ; si tous les utilisateurs visent le moins cher, il sature, et il faut alors attendre ou partager les ressources, ce qui allonge les temps. (2) Les ressources plus rapides coûtent généralement plus cher. (3) Pour aller plus vite, on loue plus de VMs, et chacune est facturée. Il s'agit d'un problème multi-objectif : on cherche un compromis (front de Pareto), par exemple minimiser le coût sous une contrainte d'échéance (deadline).

Exercice 4 : ordonnancement time-shared vs space-shared

Scénario : 1 hôte (1 PE à 1000 MIPS), 1 VM (1 PE à 1000 MIPS) et 3 cloudlets de 10 000 MI soumis en même temps (code : `Ex4Schedulers.java`).

Q1. Prédiction sur papier et diagrammes de Gantt

- **CloudletSchedulerSpaceShared** : un seul cloudlet à la fois par PE (FCFS). Chaque cloudlet dure 10 000 / 1000 = 10 s. C0 : [0 ; 10], C1 : [10 ; 20], C2 : [20 ; 30].
- **CloudletSchedulerTimeShared** : les 3 cloudlets se partagent le PE. Chacun reçoit 1000 / 3 ≈ 333 MIPS et dure 10 000 / 333 = 30 s. Tous démarrent à 0 et finissent à 30.

Gantt — CloudletSchedulerSpaceShared (1 case = 2 s)

t (s)	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28
Cloudlet 0															
Cloudlet 1															
Cloudlet 2															

Gantt — CloudletSchedulerTimeShared (1 case = 2 s ; chaque cloudlet reçoit 1/3 du PE)

t (s)	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28
Cloudlet 0	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3
Cloudlet 1	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3
Cloudlet 2	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3	1/3

Figure 1 — Diagrammes de Gantt prévus

Q2. Simulation et comparaison avec les prédictions

Ordonnanceur	Cloudlet	Début (prévu → simulé)	Fin (prévu → simulé)	Durée
SpaceShared	0	0 → 0,01	10 → 10,01	10
SpaceShared	1	10 → 10,01	20 → 20,01	10
SpaceShared	2	20 → 20,01	30 → 30,01	10
TimeShared	0	0 → 0,01	30 → 30,01	30
TimeShared	1	0 → 0,01	30 → 30,01	30
TimeShared	2	0 → 0,01	30 → 30,01	30

Tableau 8 — Prédictions vs simulation

Les résultats simulés **correspondent exactement aux prédictions**, à un décalage constant de 0,01 s près (délai de création de la VM et d'envoi des cloudlets par le broker). Le makespan est le même (30 s) dans les deux cas, car le travail total est identique. En revanche, le **temps de réponse moyen** diffère : (10 + 20 + 30) / 3 = **20 s** en space-

shared, contre **30 s** en time-shared. Le space-shared termine les premiers travaux plus tôt ; le time-shared fait progresser tout le monde en même temps.

Q3. Deux VMs de 1 PE sur cet hôte : VmSchedulerSpaceShared vs VmSchedulerTimeShared

VmScheduler de l'hôte	MIPS demandés par VM	VM 0	VM 1	Makespan
SpaceShared	1000	créée	**échec**	30,01 s (3 cloudlets sur VM 0)
TimeShared	1000	créée	**échec**	30,01 s
SpaceShared	500	créée	**échec**	60,01 s (VM 0 n'a que 500 MIPS)
TimeShared	500	créée	créée	40,01 s (C0, C2 sur VM 0 ; C1 sur VM 1)

Tableau 9 — Création de la seconde VM selon l'ordonnanceur de VMs

Qu'arrive-t-il à la seconde VM ?

- **VmSchedulerSpaceShared** : chaque PE virtuel doit recevoir un PE physique **dédié**. L'hôte n'a qu'un PE, déjà attribué à la VM 0 : la création de la VM 1 est **refusée**, quelle que soit sa puissance. Le broker essaie alors les autres datacenters (il n'y en a pas) et envoie tous les cloudlets (round-robin sur les VMs créées) vers la VM 0. Avec des VMs à 500 MIPS, c'est pire : la VM 0 n'utilise que la moitié du PE et le makespan double (60 s).
- **VmSchedulerTimeShared** : le PE peut être **partagé** entre plusieurs VMs. Mais dans CloudSim 7, ce scheduler **n'autorise pas la sur-réservation** : si chaque VM demande 1000 MIPS, 2000 > 1000 disponibles et la VM 1 est aussi refusée (le code compare `getAvailableMips() < totalRequestedMips`). Si chaque VM demande 500 MIPS, les deux VMs sont créées et se partagent le PE. Le PE entier est alors utilisé, et le makespan passe de 60 s (space-shared) à 40 s.

Pour accepter deux VMs de 1000 MIPS sur un PE de 1000 MIPS, il faut `VmSchedulerTimeSharedOverSubscription`, qui réduit proportionnellement la part de chaque VM.

Q4. Choix de politique

- **Application web interactive → time-shared.** Les requêtes sont courtes, nombreuses et arrivent en continu. Le time-shared les fait toutes progresser en même temps, si bien qu'une petite requête n'attend pas derrière une grosse : la latence perçue est faible et équitable. Il permet aussi de partager les PEs entre VMs peu chargées (meilleure utilisation, sur-réservation possible).
- **Jobs batch HPC → space-shared.** Les calculs sont longs, intensifs et souvent parallèles. Des cœurs dédiés donnent des performances prévisibles et reproductibles, sans changement de contexte ni interférence. En FCFS, les premiers travaux finissent plus tôt (temps de réponse moyen de 20 s contre 30 s dans notre test), et on peut appliquer des politiques de file d'attente (SJF, backfilling) comme sur un cluster.

Partie 2 — CloudSim Plus 8.5.7

Exercice 5 : de CloudSim 7 à CloudSim Plus

Q1. BasicFirstExample et les sept étapes d'une simulation

J'ai copié `BasicFirstExample` du dépôt `cloudsimplus-examples` (seule modification : `static void main()`, une syntaxe Java 25, est remplacée par `public static void main(String[] args)`, car nous sommes en Java 21). Voici les sept étapes vues en cours, retrouvées dans le code :

#	Étape	Code dans BasicFirstExample	Équivalent CloudSim 7
1	Initialiser le simulateur	`simulation = new CloudSimPlus();`	`CloudSim.init(numUser, cal, trace)`
2	Créer le(s) datacenter(s) et leurs hôtes (PEs)	`createDatacenter()` → `createHost()` → `new DatacenterSimple(simulation, hostList)`	`new Datacenter(name, characteristics, policy, storage, 0)`
3	Créer le broker	`broker0 = new DatacenterBrokerSimple(simulation);`	`new DatacenterBroker("Broker")`
4	Créer les VMs et les soumettre	`vmList = createVms(); broker0.submitVmList(vmList);`	`broker.submitGuestList(vmlist)`
5	Créer les cloudlets et les soumettre	`cloudletList = createCloudlets(); broker0.submitCloudletList(…)`	`broker.submitCloudletList(…)`
6	Lancer la simulation	`simulation.start();`	`CloudSim.startSimulation()` + `stopSimulation()`
7	Afficher les résultats	`new CloudletsTableBuilder(broker0.getCloudletFinishedList()).build();`	boucle `printCloudletList()` écrite à la main

Résultat : 1 hôte de 8 PEs, 2 VMs de 4 PEs et 4 cloudlets de 10 000 MI à 2 PEs. Les 4 cloudlets s'exécutent de 0,1 à **20,1 s** (2 par VM). La durée est de 20 s et non 10 s, car le modèle d'utilisation `UtilizationModelDynamic(0.5)` limite chaque cloudlet à 50 % du CPU : 10 000 MI / (1000 MIPS × 0,5) = 20 s. (Dans CloudSim Plus, la longueur est donnée par PE.)

Q2. Exercice 2 réécrit avec CloudSim Plus

Code : `Ex5Heterogeneous.java`. Mêmes hôtes, VMs et cloudlets. Le placement VM i → hôte i s'obtient sans écrire de classe, avec une simple fonction : `policy.setFindHostForVmFunction((p, vm) -> Optional.of(hosts.get((int) vm.getId())))`. L'affectation se fait par `cloudlet.setVm(vm)`. Extrait de la sortie de `CloudletsTableBuilder` :

```
                  Affectation initiale (cloudlet i -> VM i mod 4)
|Cloudlet| Status|DC|Host|Host PEs |VM|   VM
PEs|CloudletLen|FinishedLen|CloudletPEs|StartTime|FinishTime|ExecTime
|      ID|       |ID|  ID|CPU cores|ID|CPU cores|         MI|         MI|  CPU cores|  Seconds|
Seconds| Seconds
|       0|SUCCESS| 1|   0|        1| 0|        1|      30000|      30000|          1|      0.1|
60.1|    60.0
|       1|SUCCESS| 1|   1|        2| 1|        1|      60000|      60000|          1|      0.1|
40.2|    40.1
|       2|SUCCESS| 1|   2|        4| 2|        2|      90000|      90000|          1|      0.1|
45.2|    45.1
|       3|SUCCESS| 1|   3|        2| 3|        1|     120000|     120000|          1|      0.1|
40.2|    40.1
|       4|SUCCESS| 1|   0|        1| 0|        1|     150000|     150000|          1|      0.1|
180.2|   180.1
Makespan = 180.22 s          (affectation LJF : makespan = 60.21 s)
```

Q3. Comparaison des deux versions

Critère	CloudSim 7.0.1	CloudSim Plus 8.5.7
Lignes de code (hors commentaires et lignes vides)	`Ex2Heterogeneous` : 89 lignes, plus 108 lignes de classes utilitaires nécessaires (`Cs7Util`, `RecordingBroker`, `VmAllocationPolicyFixed`), soit **≈ 197 lignes**. Pour comparaison, `CloudSimExample3` fait 130 lignes pour 2 VMs et 2 cloudlets.	`Ex5Heterogeneous` : **75 lignes**, coûts et affichage de `VmCost` compris, sans aucune classe auxiliaire.
Résultats	Makespan 180,01 s (initial) et 60,01 s (LJF)	Makespan 180,22 s et 60,21 s
Lisibilité	Constructeurs à 7-9 paramètres positionnels, provisioners explicites, `try/catch` obligatoires, affichage à la main	Constructeurs courts et setters chaînés (`new VmSimple(mips, pes).setRam(…).setBw(…)`), valeurs par défaut sensées, tableau de résultats prêt à l'emploi

Les résultats sont-ils identiques ? Oui sur le fond : mêmes affectations, mêmes durées d'exécution (30, 40, 45, 40, 180 s ; puis 60 s en LJF) et même analyse. Il reste de petits écarts de 0,1 à 0,2 s. Les cloudlets démarrent à 0,1 s au lieu de 0,01 s, et les fins sont arrondies au dixième (40,2 au lieu de 40,01), car CloudSim Plus traite les événements avec un pas minimal (`minTimeBetweenEvents` = 0,1 s par défaut) et ajoute un délai de soumission.

Principales différences d'API observées :

Aspect	CloudSim 7	CloudSim Plus
Simulation	État statique global : `CloudSim.init(n, cal, trace)`, nombre d'utilisateurs à déclarer	Objet `CloudSimPlus` passé à chaque entité ; plusieurs simulations possibles dans le même programme
Paquetages / noms	`org.cloudbus.cloudsim.*` : `Host`, `Vm`, `Cloudlet`, `Datacenter`	`org.cloudsimplus.*` : interfaces + implémentations `HostSimple`, `VmSimple`, `CloudletSimple`, `DatacenterSimple`
Datacenter et coûts	`DatacenterCharacteristics` à 9 paramètres (arch, OS, VMM, hôtes, fuseau, 4 coûts) + politique + liste de stockage	`new DatacenterSimple(sim, hosts, policy)`, puis `getCharacteristics().setCostPerSecond(…)` en style fluent
Hôte	Provisioners RAM/BW/PE explicites + `VmScheduler` obligatoire	`new HostSimple(ram, bw, storage, pes)` ; `VmSchedulerSpaceShared` par défaut
VM / cloudlet	userId, VMM, taille, 3 modèles d'utilisation… ; identifiants gérés à la main	`new VmSimple(mips, pes)`, `new CloudletSimple(length, pes)` ; identifiants automatiques
Liaison cloudlet → VM	Par identifiants entiers : `setGuestId(vmId)` / `bindCloudletToVm(cId, vmId)`	Par objets : `cloudlet.setVm(vm)` / `broker.bindCloudletToVm(c, vm)`
Personnalisation	Par héritage (j'ai dû sous-classer `DatacenterBroker` et `VmAllocationPolicySimple`)	Par fonctions et lambdas : `setVmMapper`, `setFindHostForVmFunction`, écouteurs `addOnHostAllocationListener`, `addOnClockTickListener`
Résultats et traces	`Log` maison, affichage manuel	`CloudletsTableBuilder`, journalisation SLF4J/Logback avec niveaux (`Log.setLevel`)
Coûts	Par cloudlet ; `getProcessingCost()` = bande passante seule	Par VM avec `VmCost` (CPU, RAM, stockage, BW)

Q4. Coûts du datacenter et VmCost

Les coûts sont ceux de l'exercice 1 : `setCostPerSecond(3.0).setCostPerMem(0.05).setCostPerStorage(0.001).setCostPerBw(0.0)`. Chaque VM est facturée avec `new VmCost(vm)`, comme dans `CostsExample1`. D'après le code source de `VmCost` :

- CPU = (costPerSecond ÷ MIPS d'un PE de l'hôte) × MIPS totaux de la VM × durée de vie de la VM
- RAM = costPerMem × RAM de la VM ; stockage = costPerStorage × taille de l'image ; BW = costPerBw × bande passante de la VM (forfaits indépendants du temps)

a) Exercice 2 réécrit (Ex5Heterogeneous) :

VM	Affectation initiale : CPU / Total	LJF : CPU / Total	LJF + VMs détruites dès qu'inactives : CPU / Total
VM 0 (1×1000)	541,00 / 602,20 $	180,96 / 242,16 $	93,30 / 154,50 $ (vie 31,1 s)
VM 1 (1×1500)	541,00 / 602,20 $	180,96 / 242,16 $	123,63 / 184,83 $ (vie 41,2 s)
VM 2 (2×2000)	1082,00 / 1143,20 $	361,92 / 423,12 $	367,26 / 428,46 $ (vie 61,2 s)
VM 3 (1×3000)	541,00 / 602,20 $	180,96 / 242,16 $	153,63 / 214,83 $ (vie 51,2 s)
Total	**2949,81 $**	**1149,60 $**	**982,62 $**

Tableau 10 — Coût par VM (RAM = 51,20 $ et stockage = 10,00 $ pour chaque VM)

Par défaut, le broker garde **toutes les VMs actives jusqu'à la fin du dernier cloudlet** (180,33 s). La VM 3 est donc payée 180 s alors qu'elle ne travaille que 40 s : réduire le makespan avec LJF divise le coût par 2,6. En détruisant les VMs dès qu'elles deviennent inactives (`broker.setVmDestructionDelay(1.0)`), on économise encore 15 %. La VM 2 coûte deux fois plus en CPU, car elle réserve 2 PEs.

b) Scénario de l'exercice 1 refait avec CloudSim Plus (Ex5CostsEx1) : 1 PE à 1000 MIPS, 3 VMs de 250 MIPS, cloudlets de 40 000, 80 000 et 120 000 MI, 3 $/s.

	CloudSim 7 (Ex. 1)	CloudSim Plus — défaut	CloudSim Plus — VMs détruites dès qu'inactives
Temps d'exécution	160 / 320 / 480 s	160 / 320 / 480 s (identiques)	identiques
Unité facturée	le cloudlet (temps × 3 $/s)	la VM (durée de vie 480,3 s chacune)	la VM (161 / 321 / 481 s)
Coût CPU	480 + 960 + 1440 = 2880 $	3 × 360,24 = 1080,72 $	120,82 + 240,91 + 360,91 = 722,64 $
RAM + stockage	non facturés	3 × (25,60 + 10,00) = 106,80 $	106,80 $
Total	**2880 $** (et `getProcessingCost()` = 0)	**1187,52 $**	**829,45 $**

Tableau 11 — Comparaison des modèles de coût sur le scénario de l'exercice 1

Comparaison avec l'exercice 1 : les temps sont identiques, mais les coûts diffèrent fortement, car les deux simulateurs ne facturent pas la même chose :

- **CloudSim 7** facture le **cloudlet** au plein tarif de 3 $/s, même si la VM n'utilise que 250 MIPS, soit un quart du PE. Il ne compte ni la RAM ni le stockage, et `getProcessingCost()` ne renvoie que la bande passante.
- **CloudSim Plus** facture la **VM**, comme un fournisseur IaaS réel. Le prix CPU est proportionnel à la fraction de PE réservée (250/1000 × 3 = 0,75 $/s) et à la **durée de vie** de la VM, temps d'inactivité compris. La RAM et le stockage alloués sont ajoutés. On paie donc des ressources réservées, et non du travail effectué, d'où l'intérêt de détruire les VMs inactives.

Exercice 6 : politiques d'allocation de VMs

Configuration (`Ex6AllocationPolicies.java`) : 1 datacenter, 4 hôtes de **8, 4, 4 et 2 PEs** à 1000 MIPS (18 PEs, 16 Go de RAM chacun, `VmSchedulerSpaceShared` par défaut) et 8 VMs de 1 à 4 PEs. Chaque VM exécute un cloudlet qui utilise tous ses PEs pendant 20 s. J'ai désactivé les nouvelles tentatives de création de VM (`getVmCreation().setRetryDelay(-1)`) : sinon, le broker CloudSim Plus réessaie toutes les 5 s de créer les VMs refusées et la simulation ne se termine jamais.

J'ai testé trois jeux de tailles, qui « rentrent » tous en théorie (le total ne dépasse pas 18 PEs et un placement complet existe) : 16 PEs (marge), 18 PEs (plein, sans marge) et 12 PEs (charge légère). Notation : `H0[1:4, 5:1]` signifie que l'hôte 0 porte la VM 1 (4 PEs) et la VM 5 (1 PE).

Q1 et Q2. Résultats

Scénario 1 — VMs = [2, 4, 1, 3, 2, 1, 1, 2] PEs → 16 / 18 PEs

Politique	Hôtes utilisés	VMs placées sur chaque hôte (VM:PEs)	VMs non créées	Makespan
Simple	4	H0[0:2,1:4,5:1] H1[2:1,4:2] H2[3:3] H3[6:1]	**VM 7**	20,10 s (7/8)
FirstFit	4	H0[0:2,1:4,2:1] H1[3:3] H2[4:2,5:1,6:1] H3[7:2]	aucune	20,10 s
BestFit	4	H0[4:2,5:1,6:1,7:2] H1[1:4] H2[2:1,3:3] H3[0:2]	aucune	20,10 s
RoundRobin	4	H0[0:2,3:3,6:1,7:2] H1[1:4] H2[2:1,4:2] H3[5:1]	aucune	20,10 s

Scénario 2 — VMs = [2, 4, 1, 3, 2, 1, 3, 2] PEs → 18 / 18 PEs

Politique	Hôtes utilisés	VMs placées sur chaque hôte (VM:PEs)	VMs non créées	Makespan
Simple	4	H0[0:2,1:4,5:1] H1[2:1,4:2] H2[3:3] H3[7:2]	**VM 6**	20,10 s (7/8)
FirstFit	4	H0[0:2,1:4,2:1] H1[3:3] H2[4:2,5:1] H3[7:2]	**VM 6**	20,10 s (7/8)
BestFit	4	H0[4:2,5:1,6:3,7:2] H1[1:4] H2[2:1,3:3] H3[0:2]	aucune	20,10 s (8/8)
RoundRobin	4	H0[0:2,3:3,6:3] H1[1:4] H2[2:1,4:2] H3[5:1]	**VM 7**	20,10 s (7/8)

Scénario 3 — VMs = [2, 1, 3, 1, 2, 1, 1, 1] PEs → 12 / 18 PEs (charge légère)

Politique	Hôtes utilisés	VMs placées sur chaque hôte (VM:PEs)	VMs non créées	Makespan
Simple	3	H0[0:2,1:1,2:3,6:1] H1[3:1,5:1,7:1] H2[4:2] H3[ ]	aucune	20,10 s
FirstFit	3	H0[0:2,1:1,2:3,3:1] H1[4:2,5:1,6:1] H2[7:1] H3[ ]	aucune	20,10 s
BestFit	4	H0[6:1,7:1] H1[1:1,2:3] H2[3:1,4:2,5:1] H3[0:2]	aucune	20,10 s
RoundRobin	4	H0[0:2,4:2] H1[1:1,5:1] H2[2:3,6:1] H3[3:1,7:1]	aucune	20,10 s

Tableau 12 — Résultats des 4 politiques (outputs/Ex6AllocationPolicies.txt)

Fonctionnement observé de chaque politique (code source CloudSim Plus 8.5.7)

- **VmAllocationPolicySimple** (par défaut) : choisit l'hôte qui a le **plus de PEs libres** (worst fit). Elle étale les VMs et laisse un peu de place libre partout.
- **VmAllocationPolicyFirstFit** : choisit le premier hôte qui convient, en reprenant à partir du dernier hôte utilisé (en pratique un « next fit »).
- **VmAllocationPolicyBestFit** : choisit l'hôte qui a le **moins de PEs libres** parmi ceux qui conviennent. Elle remplit les hôtes au plus juste.
- **VmAllocationPolicyRoundRobin** : passe à l'hôte suivant à chaque VM, de façon circulaire.

Q3. Analyse : économie d'énergie ou répartition de la charge ?

Makespan : il vaut 20,10 s dans tous les cas. Avec `VmSchedulerSpaceShared`, chaque VM obtient des PEs dédiés : le placement ne change pas la vitesse d'exécution tant qu'il n'y a pas de sur-réservation. La vraie différence porte sur le **nombre de VMs créées** (le cloudlet d'une VM refusée n'est jamais exécuté : 7/8) et sur le **nombre d'hôtes allumés**.

- **Fragmentation :** en étalant les VMs, Simple laisse 4 PEs libres répartis à raison de 1 par hôte (scénario 1). La VM 7, qui demande 2 PEs, est alors refusée alors qu'il reste globalement de la place. En charge pleine (scénario 2), **seule BestFit place les 8 VMs** : en remplissant les hôtes au plus juste, elle garde des blocs libres contigus.
- **Économie d'énergie (consolidation) → BestFit**, la politique de consolidation par principe. Elle concentre les VMs sur les hôtes les plus remplis, ce qui permet d'éteindre ou de mettre en veille les autres, et c'est la seule qui ne gaspille pas de capacité quand le datacenter est plein. Nuance importante : dans le scénario 3, BestFit allume 4 hôtes contre 3 pour Simple et FirstFit. Placée en ligne, la première VM tombe sur le plus petit hôte (H3, 2 PEs), qui est alors « le mieux ajusté ». Pour vraiment minimiser le nombre d'hôtes actifs, il faut trier les VMs par taille décroissante et privilégier les grands hôtes (Best/First Fit Decreasing), puis consolider dynamiquement par migration (les politiques `…Migration…` de CloudSim).

- **Répartition de la charge → RoundRobin** (et, dans une moindre mesure, **Simple**). RoundRobin distribue les VMs uniformément (2 VMs par hôte dans le scénario 3). Simple équilibre les PEs libres et évite de surcharger un hôte. Ces politiques limitent les points chauds et la contention, et améliorent la tolérance aux pannes, mais au prix de plus d'hôtes allumés et d'une fragmentation qui peut faire refuser des VMs.

Exercice 7 : brokers et répartition de charge

Q1. Algorithmes de broker et de load balancing présents dans les exemples

Oui, les deux frameworks en contiennent, mais avec des philosophies différentes. CloudSim 7 propose surtout des politiques de placement et de consolidation (énergie), plus des load balancers « web ». CloudSim Plus propose plusieurs brokers prêts à l'emploi et des mécanismes de mise à l'échelle (scaling).

CloudSim 7.0.1

Classe(s) / exemple	Fonctionnement
`DatacenterBroker` (CloudSimExample1 à 9)	Broker de base. VMs : toutes demandées au **premier** datacenter de la liste, puis au suivant pour celles qui échouent. Cloudlets : **round-robin** sur les VMs créées (`guestIndex = (guestIndex+1) % nbVMs`), sauf si le cloudlet est lié à une VM (`setGuestId`).
`CloudSimExample8` (GlobalBroker)	Entité qui crée un nouveau broker pendant la simulation (arrivée dynamique d'utilisateurs). Ce n'est pas un algorithme d'équilibrage.
`VmAllocationPolicySimple` + `SelectionPolicyLeastFull`	Placement des VMs sur l'hôte **le moins rempli** (le plus de PEs libres) : étale la charge entre hôtes.
`VmAllocationWithSelectionPolicy` + `SelectionPolicyFirstFit / MostFull / LeastFull / WorstFit / RandomSelection / MinimumUtilization…`	Placement paramétré par une politique de sélection d'hôte : premier qui convient, plus rempli (consolidation), moins rempli (équilibrage), aléatoire…
Exemples `power/planetlab` et `power/random` (ThrMmt, IqrMc, MadMu, LrRs, LrrMmt…)	Consolidation dynamique avec migration. `PowerVmAllocationPolicyMigrationStaticThreshold / InterQuartileRange / MedianAbsoluteDeviation / LocalRegression / LocalRegressionRobust` détectent les hôtes **surchargés** (seuil fixe ou adaptatif). Une politique de sélection (`SelectionPolicyMinimumMigrationTime`, `PowerSelectionPolicyMaximumCorrelation`, `SelectionPolicyMinimumUtilization`, `SelectionPolicyRandomSelection`) choisit les VMs à migrer. Elles sont replacées par un best-fit décroissant sensible à l'énergie, et les hôtes sous-utilisés sont vidés puis éteints.
Exemples `web` : `WebBroker` + `SimpleWebLoadBalancer`, `RoundRobinWebLoadBalancer`, `SimpleDBBalancer`, `RoundRobinDBBalancer`	Véritables load balancers applicatifs. Les sessions web sont envoyées au serveur d'application **le moins chargé en CPU** (Simple) ou à tour de rôle (RoundRobin) ; même principe pour les serveurs de base de données.
Paquetage `container` (ContainerDatacenterBroker…)	Placement et migration de conteneurs dans des VMs, avec les mêmes familles de politiques.

CloudSim Plus 8.5.7

Classe(s) / exemple	Fonctionnement
`DatacenterBrokerSimple`	Cloudlet → VM en **round-robin** sur les VMs en exécution. Datacenter : passe au suivant si le courant ne peut pas créer toutes les VMs, ou choisit le plus proche par fuseau horaire avec `setSelectClosestDatacenter(true)` (DatacenterSelectionByTimeZoneExample).
`DatacenterBrokerFirstFit`	Première VM (parcours circulaire) ayant assez de PEs libres « attendus » pour le cloudlet.
`DatacenterBrokerBestFit` (CloudletToVmMappingBestFit, DatacenterBrokersMappingComparison)	VM ayant **le moins de PEs libres** parmi celles qui peuvent recevoir le cloudlet : remplit les VMs au plus juste.
`DatacenterBrokerHeuristic` + `CloudletToVmMappingSimulatedAnnealing` (DatacenterBrokerHeuristicExample)	Recherche une bonne affectation globale cloudlets → VMs par **recuit simulé** : on part d'une solution aléatoire, on la perturbe et on accepte parfois une moins bonne solution pour sortir des optimums locaux.
`setVmMapper(…)`, sélection du datacenter (SelectNextDatacenterExample, BrokerTargetDatacenterExample)	Remplacer l'algorithme du broker par une simple fonction lambda, sans sous-classe.

Classe(s) / exemple	Fonctionnement
`LoadBalancerByHorizontalVmScalingExample` (`HorizontalVmScalingSimple`)	**Équilibrage par mise à l'échelle horizontale** : quand une VM est surchargée (prédicat sur l'utilisation CPU), une nouvelle VM est créée pour absorber les cloudlets qui arrivent ; les VMs inactives sont détruites après un délai.
`VerticalVmCpuScalingExample`, `VerticalVmRamScalingExample`	Mise à l'échelle verticale : ajout ou retrait de PEs ou de RAM à une VM selon des seuils.
`MigrationExample1` (`VmAllocationPolicyMigrationBestFitStaticThreshold`)	Rééquilibrage par migration : les VMs des hôtes au-dessus d'un seuil d'utilisation sont déplacées vers des hôtes moins chargés (best fit).
`VmAllocationPolicyRoundRobinExample`, `RandomVmAllocationPolicyExample`	Répartition des VMs entre hôtes à tour de rôle ou au hasard.

Q2. DatacenterBrokerSimple vs DatacenterBrokerBestFit

Configuration (`Ex7Brokers.java`) : 1 hôte de 4 PEs à 4000 MIPS, 3 VMs de 1 PE à **1000, 2000 et 4000 MIPS** et 12 cloudlets de 1 PE dont la longueur est tirée au hasard dans [10 000 ; 100 000] MI avec `new Random(42)` :

```
[23770, 41459, 24946, 87741, 32604, 55047, 34894, 50032, 49639, 15914, 15627, 36735]  (somme = 468 408 MI)
```

Borne inférieure du makespan : 468 408 ÷ (1000 + 2000 + 4000) = **66,92 s**.

Broker	Cloudlets exécutés	Cloudlets par VM (1000 / 2000 / 4000 MIPS)	Makespan
DatacenterBrokerSimple	12 / 12	4 / 4 / 4	**162,75 s**
DatacenterBrokerBestFit (tel quel)	**3 / 12**	1 / 1 / 1	23,94 s (non comparable)
DatacenterBrokerBestFit + ressoumission de la file d'attente	12 / 12	2 / 3 / 7	**79,20 s**
DatacenterBrokerFirstFit + ressoumission (bonus)	12 / 12	2 / 3 / 7	79,20 s
Simple + mapper MCT personnalisé (bonus)	12 / 12	2 / 4 / 6	**73,89 s**

Tableau 13 — Comparaison des brokers (outputs/Ex7Brokers.txt)

Analyse :

- **DatacenterBrokerSimple** fait un round-robin aveugle : 4 cloudlets par VM, **sans tenir compte des MIPS**. La VM de 1000 MIPS reçoit 162 319 MI (dont le cloudlet de 87 741 MI) et finit à 162,75 s, tandis que la VM de 4000 MIPS a terminé à 41,8 s. Le makespan est 2,4 fois la borne inférieure.
- **DatacenterBrokerBestFit utilisé tel quel n'exécute que 3 cloudlets sur 12.** Il place un cloudlet seulement sur une VM qui a un PE « attendu » libre (`getExpectedFreePesNumber() >= pes`). Avec des VMs de 1 PE, les 3 premiers cloudlets occupent les 3 PEs et les 9 autres sont mis en attente (« postponed »). Dans CloudSim Plus 8.5.7, la liste d'attente n'est réexaminée qu'à la création de nouvelles VMs, pas à la fin d'un cloudlet : ces 9 cloudlets ne sont donc jamais exécutés. Le « makespan » de 23,94 s est trompeur. BestFit est conçu pour des VMs à plusieurs PEs et des cloudlets qui arrivent dynamiquement.
- **Comparaison équitable :** j'ai ajouté un écouteur `simulation.addOnClockTickListener` qui ressoumet la liste d'attente dès qu'un PE de VM se libère. BestFit se comporte alors comme une file d'attente avec dispatching au premier PE libre : chaque cloudlet part sur la première VM disponible, et la VM rapide, qui se libère plus souvent, en reçoit 7. **Makespan : 79,20 s, soit −51 % par rapport à Simple.** Il n'est qu'à 18 % de la borne inférieure, car les cloudlets ne sont pas triés (le cloudlet de 55 047 MI tombe sur la VM lente à t = 24 s).
- **Pour aller plus loin :** un mapper « MCT » (Minimum Completion Time) écrit en 10 lignes avec `broker.setVmMapper(…)` choisit pour chaque cloudlet la VM où il finira le plus tôt, compte tenu des MI déjà affectés. Il obtient **73,89 s** et répartit la charge en proportion des MIPS (2 / 4 / 6). Trier d'abord les cloudlets par longueur décroissante (LJF + MCT, comme dans l'exercice 2) rapprocherait encore de la borne.

Conclusion de l'exercice : sur des VMs hétérogènes, un broker qui ignore la puissance des VMs (round-robin) donne un mauvais makespan. Les brokers « fit » raisonnent en PEs libres et non en MIPS ; ils n'équilibrent la

charge que si les cloudlets sont redistribués au fil de l'eau. Le meilleur résultat vient d'une heuristique qui tient compte à la fois de la longueur des tâches et de la vitesse des VMs.

Conclusion

Ce TP m'a permis de manipuler les deux simulateurs et d'observer concrètement plusieurs mécanismes du cloud :

- **Coûts** : CloudSim 7 facture le cloudlet, et son `getProcessingCost()` ne contient que la bande passante, un piège à connaître. CloudSim Plus facture la VM (ressources réservées, temps d'inactivité compris), ce qui est plus proche d'un fournisseur IaaS réel.
- **Ordonnancement** : l'affectation des cloudlets aux VMs pèse beaucoup sur le makespan : 180 → 60 s avec LJF (Ex. 2), 162,75 → 73,89 s avec MCT (Ex. 7). Le choix time-shared / space-shared arbitre entre réactivité (web) et prévisibilité (HPC).
- **Placement** : le broker CloudSim 7 remplit le premier datacenter sans regarder le prix ; un broker sensible au coût divise la facture par deux (Ex. 3). Les politiques d'allocation de VMs arbitrent entre consolidation (BestFit, énergie) et étalement (RoundRobin/Simple, répartition de charge), avec un risque de fragmentation.
- **Outils** : CloudSim Plus est nettement plus concis (≈ 75 lignes contre ≈ 197) et plus extensible grâce aux lambdas et aux écouteurs, mais il faut connaître son comportement par défaut : nouvelles tentatives infinies de création de VM, cloudlets en attente non ressoumis avec BestFit, VMs gardées jusqu'à la fin.

Toutes les sorties brutes sont dans `TP3/outputs/` et le code est prêt à être réexécuté avec les commandes indiquées en introduction.